In the context of project management, we have come to believe CVS is essential. It allows developers to collaborate efficiently, even when they are in separate locations. Popular Net projects such as Apache, Linux, and PHP use CVS and probably would not be as successful if they did not.

Modularization Using include

Despite its name, the include function is not equivalent to C's preprocessor command of the same name. In many ways it is like a function call. Given the name of a file, PHP attempts to parse the file as if it appeared in place of the call to include. The difference from a function is that the code will be parsed only if the include statement is executed. You can take advantage of this by wrapping calls to include in if statements. The require function, on the other hand, will always include the specified file, even if it is inside an if block that is never executed. It has been discussed several times on the PHP mailing list that require is faster than include because PHP is able to inject the specified file into the script during an early pass across the code. However, this applies only to files specified by a static path. If the call to require contains a variable, it cannot be executed until the runtime. It may be helpful to adopt a rule of using require only when outside a compound statement and when specifying a static path.

Almost anything we write in PHP uses include extensively. The first reason is that it makes the code more readable. But the other reason is that it breaks the site into modules. This allows multiple people to work on the site at once. It forces you to write code that is more easily reused, within the existing site and on your next project. Most Web sites have to rely on repeating elements. Consistent navigation aids the user, but it is also a major problem when building and maintaining the site. Each page has to have a duplicate code block pasted into it. Making this a module and including it allows you to debug the code once, making changes quickly.

You can adopt a strategy that consists of placing functions into include modules. As each script requires a particular function, you can simply add an include. If your library of functions is small enough, you might place them all into one file. However, you likely will have pieces of code that are needed on just a handful of pages. In this case, you'll want this module to stand alone.

As your library of functions grows, you may discover some interdependencies. Imagine a module for establishing a connection to a database, plus a couple of other modules that rely on the database connection. Each of these two scripts will include the database connection module. But what happens when both are themselves included in a script? The database module is included twice. This may cause a second connection to be made to the database, and if any functions are defined, PHP will report the error of a duplicate function.

In C, programmers avoid this situation by defining constants inside the included files, and you can adopt a similar strategy. You can define a constant inside your